

End Term Project Report

On

Introduction to Databases (CSE 3151)

Submitted by

Name : Nishant Raj
Reg. No. : 2341018028
Branch : Computer Science & Engineering
Semester : 06
Section : 23412N3
Session : 2025-2026
Admission Batch : 2023



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
FACULTY OF ENGINEERING & TECHNOLOGY (ITER)
SIKSHA 'O' ANUSANDHAN DEEMED TO BE UNIVERSITY
BHUBANESWAR, ODISHA – 751030

TABLE OF CONTENTS

Section No.	Topic	Page No.
01	Objective & Introduction	1
02	Technology Stack	2
03	Database Design and Schema	3
04	System Functionalities	4
05	Technical Implementation	5
06	Test Cases and Execution Results	6
07	Conclusion	9
	Annexure (SQL & Java)	10

1. Objective & Introduction

1.1 Objective

The primary objective of this project is to design and implement a miniature University Management System integrating a Java menu-driven application with a MySQL backend database using JDBC connectivity. This fulfils the Course Outcome (CO2) to model SQL queries and develop a database application as per user requirements.

1.2 Introduction

A university database system maintains crucial information regarding instructors, students, departments, courses, sections, and enrolment details. This project translates the standard university database schema into a functional application. Through a Command Line Interface (CLI), users can perform essential CRUD (Create, Read, Update, Delete) operations, manage academic records, and execute business logic such as calculating standard financial adjustments (HRA deposits and TDS deductions) for university personnel.

2. Technology Stack

- i. **Backend Database:** MySQL Server 8.0 (Relational Database Management System)
- ii. **Frontend Application:** Java Development Kit (JDK)
- iii. **Database Connectivity:** JDBC API utilizing MySQL Connector/J driver
- iv. **Interface:** Java Command Line Interface (CLI) via `java.util.Scanner`

3. Database Design & Schema

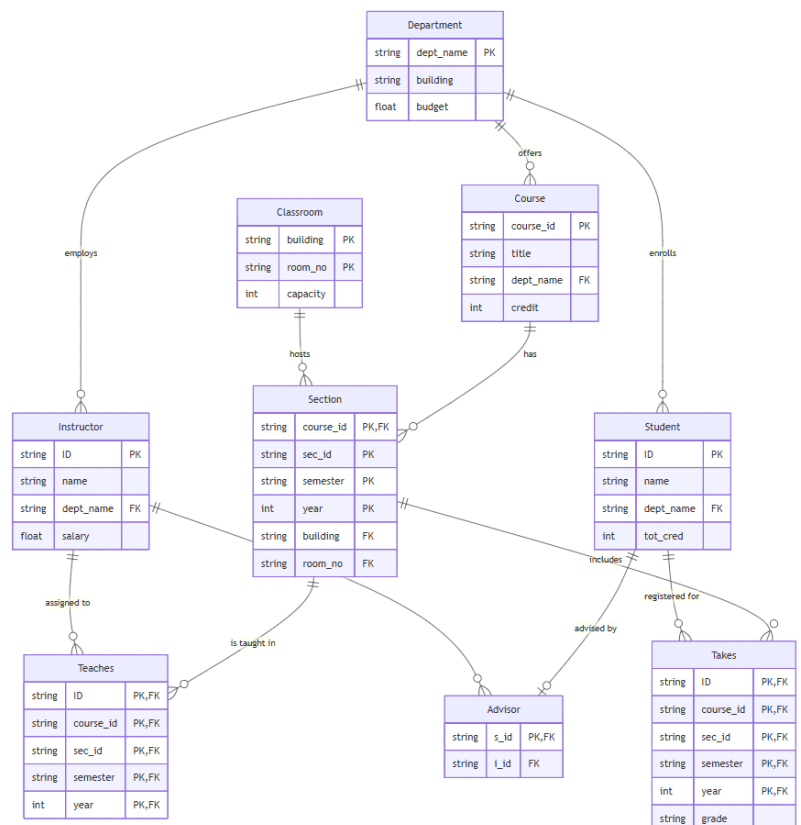
The system utilizes a standard 9-table schema strictly adhering to academic integrity rules and referential constraints.

3.1 Core Entities

- i. **Department:** Stores department name, building, and budget.
Constraint: Budget must be strictly greater than 0.
- ii. **Instructor:** Stores 5-character ID, name, department, and salary.
Constraint: Salary must exceed 20,000.
- iii. **Student:** Stores 5-character ID, name, department, and total credits.
Constraint: Total credits cannot be negative.
- iv. **Course & Section:** Manages curriculum and schedule offerings.
Semesters are restricted to 'Spring', 'Summer', 'Fall', or 'Winter'.

3.2 Relationship Mapping

Referential integrity is maintained via Foreign Keys across the schema. The teaches table maps Instructors to Sections, while the takes table tracks Student enrolments and grades. The schema restricts data anomalies; for example, a student cannot be assigned to a non-existent department.



4. System Functionalities

The application implements a 10-option menu-driven system processing the following operations:

- i. **Data Retrieval:** Dynamically fetching and displaying formatted records for students, instructors, and comprehensive course enrollments using JOIN operations.
- ii. **Data Modification:** Inserting new entities (e.g., admitting a student) while respecting 5-character ID limits, and updating or deleting existing records.
- iii. **Financial Calculations:** HRA Deposit (15% bonus) and TDS Deduction (20% tax) applied directly to instructor salaries.

5. Technical Implementation

5.1 JDBC Connectivity

The application establishes a secure connection to the local MySQL instance using the `DriverManager.getConnection()` method, authenticating via the root user and password.

5.2 Query Execution Security

To prevent SQL injection and enhance query execution efficiency, the application extensively uses `PreparedStatement` for all dynamic queries involving user input. Standard Statement objects are reserved solely for static, parameter-less retrieval.

5.3 Exception Handling

Database constraints are rigorously enforced. For instance, attempting to insert an ID exceeding 5 characters throws a `SQLException` (Data truncation error). The Java program intercepts these exceptions using try-catch blocks to prevent application crashes and gracefully notify the user.

6. Test Cases & Execution Results

The system was successfully validated against the following assignment test parameters:

Test Case	Action Performed	Observed Result
Insert Student	Added <12346, Swati Singh, History, 50>;	Record successfully inserted into student table.
Constraint Check	Attempted ID 123456 (6 digits)	Rejected by MySQL: Data too long for column 'ID'. Handled safely.
HRA Calculation	Option 8 on Instructor 10101 (65000)	Salary correctly updated to 74750.00 (15% increase).
TDS Calculation	Option 9 on Instructor 12121 (90000)	Salary correctly reduced to 72000.00 (20% decrease).
Data Deletion	Option 3 on Student 70557	Record explicitly purged from the database.

```
Windows PowerShell
PS C:\Users\ASUS> cd C:\Users\ASUS\OneDrive\Desktop\DB_Project
PS C:\Users\ASUS\OneDrive\Desktop\DB_Project> javac jdbcproject.java
PS C:\Users\ASUS\OneDrive\Desktop\DB_Project> java -cp ".;mysql-connector-j-9.7.0.jar" jdbcproject
Connection failed: Access denied for user 'root'@'localhost' (using password: YES)
PS C:\Users\ASUS\OneDrive\Desktop\DB_Project> javac jdbcproject.java
PS C:\Users\ASUS\OneDrive\Desktop\DB_Project> java -cp ".;mysql-connector-j-9.7.0.jar" jdbcproject
Connected to the database!

***** University Management System *****
1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program
Enter your choice(1-10): 1
ID      | Name      | Dept      | Credits
00128   | Zhang     | Comp. Sci. | 102
12345   | Shankar   | Comp. Sci. | 32
19991   | Brandt    | History    | 80
23121   | Chavez    | Finance    | 110
44553   | Peltier   | Physics    | 56
45678   | Levy      | Physics    | 46
54321   | Williams  | Comp. Sci. | 54
55739   | Sanchez   | Music      | 38
70557   | Snow      | Physics    | 0
76543   | Brown     | Comp. Sci. | 58
76653   | Aoi       | Elec. Eng. | 60
98765   | Bourikas  | Elec. Eng. | 98
98988   | Tanaka    | Biology    | 120

***** University Management System *****
1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
```


Enter your choice(1-10): 1

ID	Name	Dept	Credits
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

Enter your choice(1-10): 2

Enter ID: 12346

Enter Name: Swati Singh

Enter Dept: History

Enter Credits: 50

Student added.

ID	Name	Dept	Credits
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
12346	Swati Singh	History	50
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

***** University Management System *****

1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program

```
Enter your choice(1-10): 8
Enter Instructor ID: 10101
HRA processed.
ID: 10101 | Name: Srinivasan | Salary: 74750.00
```

```
***** University Management System *****
1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program
```

```
Enter your choice(1-10): 9
Enter Instructor ID: 12121
TDS processed.
ID: 12121 | Name: Wu | Salary: 72000.00
```

```
***** University Management System *****
1. Show Student Records
2. Add Student Record
3. Delete Student Record
4. Update Student Information
5. Show Instructor Details
6. Show Course Details with Enrolled Students
7. Show Course Details taken by Instructor
8. Deposit HRA To Salary
9. Deduct TDS From Salary
10. EXIT The Program
```

```
Enter your choice(1-10): 10
Exiting...
PS C:\Users\ASUS\OneDrive\Desktop\DB_Project> |
```

7. Conclusion

The Mini Project was successfully executed, fulfilling all the requirements of Laboratory Assignments 4 and 6. The integration between the Java application and the MySQL database operates flawlessly, effectively managing CRUD operations, ensuring strict data integrity via constraints, and computing dynamic algorithmic updates. This effectively demonstrates the practical application of Database System Concepts and JDBC standard practices.

Annexure

SQL

-- 1. Insert Departments

INSERT INTO department VALUES

('Biology', 'Watson', 90000), ('Comp. Sci.', 'Taylor', 100000),
('Elec. Eng.', 'Taylor', 85000), ('Finance', 'Painter', 120000),
('History', 'Painter', 50000), ('Music', 'Packard', 80000),
('Physics', 'Watson', 70000);

-- 2. Insert Classrooms

INSERT INTO classroom VALUES

('Packard', '101', 500), ('Painter', '514', 10),
('Taylor', '3128', 70), ('Watson', '100', 30), ('Watson', '120', 50);

-- 3. Insert Courses

INSERT INTO course VALUES

('BIO-101', 'Intro. to Biology', 'Biology', 4), ('BIO-301', 'Genetics', 'Biology', 4),
('BIO-399', 'Computational Biology', 'Biology', 3), ('CS-101', 'Intro. to Computer Science',
'Comp. Sci.', 4),
('CS-190', 'Game Design', 'Comp. Sci.', 4), ('CS-315', 'Robotics', 'Comp. Sci.', 3),
('CS-319', 'Image Processing', 'Comp. Sci.', 3), ('CS-347', 'Database System Concepts', 'Comp.
Sci.', 3),
('EE-181', 'Intro. to Digital Systems', 'Elec. Eng.', 3), ('FIN-201', 'Investment Banking', 'Finance',
3),
('HIS-351', 'World History', 'History', 3), ('MU-199', 'Music Video Production', 'Music', 3),
('PHY-101', 'Physical Principles', 'Physics', 4);

-- 4. Insert Instructors

INSERT INTO instructor VALUES

('10101', 'Srinivasan', 'Comp. Sci.', 65000), ('12121', 'Wu', 'Finance', 90000),
('15151', 'Mozart', 'Music', 40000), ('22222', 'Einstein', 'Physics', 95000),
('32343', 'El Said', 'History', 60000), ('33456', 'Gold', 'Physics', 87000),
('45565', 'Katz', 'Comp. Sci.', 75000), ('58583', 'Califieri', 'History', 62000),
('76543', 'Singh', 'Finance', 80000), ('76766', 'Crick', 'Biology', 72000),
('83821', 'Brandt', 'Comp. Sci.', 92000), ('98345', 'Kim', 'Elec. Eng.', 80000);

-- 5. Insert Students

INSERT INTO student VALUES

('00128', 'Zhang', 'Comp. Sci.', 102), ('12345', 'Shankar', 'Comp. Sci.', 32),
('19991', 'Brandt', 'History', 80), ('23121', 'Chavez', 'Finance', 110),
('44553', 'Peltier', 'Physics', 56), ('45678', 'Levy', 'Physics', 46),
('54321', 'Williams', 'Comp. Sci.', 54), ('55739', 'Sanchez', 'Music', 38),
('70557', 'Snow', 'Physics', 0), ('76543', 'Brown', 'Comp. Sci.', 58),
('76653', 'Aoi', 'Elec. Eng.', 60), ('98765', 'Bourikas', 'Elec. Eng.', 98),
('98988', 'Tanaka', 'Biology', 120);

-- 6. Insert Sections

INSERT INTO section VALUES

('BIO-101', '1', 'Summer', 2009, 'Painter', '514'), ('BIO-301', '1', 'Summer', 2010, 'Painter', '514'),
('CS-101', '1', 'Fall', 2009, 'Packard', '101'), ('CS-101', '1', 'Spring', 2010, 'Packard', '101'),
('CS-190', '1', 'Spring', 2009, 'Taylor', '3128'), ('CS-190', '2', 'Spring', 2009, 'Taylor', '3128'),
('CS-315', '1', 'Spring', 2010, 'Watson', '120'), ('CS-319', '1', 'Spring', 2010, 'Watson', '100'),
('CS-319', '2', 'Spring', 2010, 'Taylor', '3128'), ('CS-347', '1', 'Fall', 2009, 'Taylor', '3128'),
('EE-181', '1', 'Spring', 2009, 'Taylor', '3128'), ('FIN-201', '1', 'Spring', 2010, 'Packard', '101'),
('HIS-351', '1', 'Spring', 2010, 'Painter', '514'), ('MU-199', '1', 'Spring', 2010, 'Packard', '101'),
('PHY-101', '1', 'Fall', 2009, 'Watson', '100');

-- 7. Insert Teaches

INSERT INTO teaches VALUES

('10101', 'CS-101', '1', 'Fall', 2009), ('10101', 'CS-315', '1', 'Spring', 2010),
('10101', 'CS-347', '1', 'Fall', 2009), ('12121', 'FIN-201', '1', 'Spring', 2010),
('15151', 'MU-199', '1', 'Spring', 2010), ('22222', 'PHY-101', '1', 'Fall', 2009),
('32343', 'HIS-351', '1', 'Spring', 2010), ('45565', 'CS-101', '1', 'Spring', 2010),
('45565', 'CS-319', '1', 'Spring', 2010), ('76766', 'BIO-101', '1', 'Summer', 2009),
('76766', 'BIO-301', '1', 'Summer', 2010), ('83821', 'CS-190', '1', 'Spring', 2009),
('83821', 'CS-190', '2', 'Spring', 2009), ('83821', 'CS-319', '2', 'Spring', 2010),
('98345', 'EE-181', '1', 'Spring', 2009);

-- 8. Insert Takes

INSERT INTO takes VALUES

('00128', 'CS-101', '1', 'Fall', 2009, 'A'), ('00128', 'CS-347', '1', 'Fall', 2009, 'A-'),
('12345', 'CS-101', '1', 'Fall', 2009, 'C'), ('12345', 'CS-190', '2', 'Spring', 2009, 'A'),
('12345', 'CS-315', '1', 'Spring', 2010, 'A'), ('12345', 'CS-347', '1', 'Fall', 2009, 'A'),
('19991', 'HIS-351', '1', 'Spring', 2010, 'B'), ('23121', 'FIN-201', '1', 'Spring', 2010, 'C+'),
('44553', 'PHY-101', '1', 'Fall', 2009, 'B-'), ('45678', 'CS-101', '1', 'Fall', 2009, 'F'),
('45678', 'CS-101', '1', 'Spring', 2010, 'B+'), ('45678', 'CS-319', '1', 'Spring', 2010, 'B'),
('54321', 'CS-101', '1', 'Fall', 2009, 'A-'), ('54321', 'CS-190', '2', 'Spring', 2009, 'B+');

Java

```
import java.sql.*;
import java.util.Scanner;

public class jdbcproject {
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        // Update with your actual database name and password
        String url = "jdbc:mysql://localhost:3306/university";
        String user = "root";
        String password = "YOUR_PASSWORD_HERE"; // <-- CHANGE THIS TO YOUR
        MYSQL PASSWORD

        try (Connection conn = DriverManager.getConnection(url, user, password)) {
            int ch;
            if (conn != null) {
                System.out.println("Connected to the database!");
            }

            do {
                System.out.println("\n\n***** University Management System *****");
                System.out.println("1. Show Student Records");
                System.out.println("2. Add Student Record");
                System.out.println("3. Delete Student Record");
                System.out.println("4. Update Student Information");
                System.out.println("5. Show Instructor Details");
                System.out.println("6. Show Course Details with Enrolled Students");
```

```
System.out.println("7. Show Course Details taken by Instructor");
System.out.println("8. Deposit HRA To Salary");
System.out.println("9. Deduct TDS From Salary");
System.out.println("10. EXIT The Program");
System.out.print("Enter your choice(1-10): ");
```

```
ch = sc.nextInt();
sc.nextLine(); // Consume newline
```

```
switch (ch) {
    case 1:
        showStudents(conn);
        break;
    case 2:
        addStudent(conn);
        showStudents(conn);
        break;
    case 3:
        deleteStudent(conn);
        showStudents(conn);
        break;
    case 4:
        updateStudent(conn);
        showStudents(conn);
        break;
    case 5:
        System.out.print("Enter Instructor ID: ");
        showInstructor(conn, sc.nextLine());
```



```
        break;
    case 6:
        showCourseEnrollment(conn);
        break;
    case 7:
        showInstructorCourses(conn);
        break;
    case 8:
        updateSalary(conn, "HRA");
        break;
    case 9:
        updateSalary(conn, "TDS");
        break;
    case 10:
        System.out.println("Exiting...");
        break;
    default:
        System.out.println("Please enter valid input (1-10)");
    }
} while (ch != 10);

conn.close();

} catch (SQLException e) {
    System.out.println("Connection failed: " + e.getMessage());
}
}
```

```
// --- Database Operation Methods ---
```

```
private static void showStudents(Connection conn) throws SQLException {  
    String query = "SELECT * FROM student";  
    try (Statement stmt = conn.createStatement(); ResultSet rs = stmt.executeQuery(query)) {  
        System.out.printf("%-10s | %-20s | %-20s | %-10s\n", "ID", "Name", "Dept", "Credits");  
        while (rs.next()) {  
            System.out.printf("%-10s | %-20s | %-20s | %-10d\n",  
                rs.getString("ID"), rs.getString("name"), rs.getString("dept_name"),  
                rs.getInt("tot_cred"));  
        }  
    }  
}
```

```
private static void addStudent(Connection conn) throws SQLException {  
    System.out.print("Enter ID: "); String id = sc.nextLine();  
    System.out.print("Enter Name: "); String name = sc.nextLine();  
    System.out.print("Enter Dept: "); String dept = sc.nextLine();  
    System.out.print("Enter Credits: "); int cred = sc.nextInt();  
    String sql = "INSERT INTO student VALUES (?, ?, ?, ?)";  
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {  
        pstmt.setString(1, id); pstmt.setString(2, name);  
        pstmt.setString(3, dept); pstmt.setInt(4, cred);  
        pstmt.executeUpdate();  
        System.out.println("Student added.");  
    }  
}
```

```
private static void deleteStudent(Connection conn) throws SQLException {
```

```

        System.out.print("Enter ID to delete: ");
        String id = sc.nextLine();

        try (PreparedStatement pstmt = conn.prepareStatement("DELETE FROM student WHERE
ID = ?")) {
            pstmt.setString(1, id);
            pstmt.executeUpdate();
            System.out.println("Record deleted.");
        }
    }
}

```

```

private static void updateStudent(Connection conn) throws SQLException {
    System.out.print("Enter Student ID: "); String id = sc.nextLine();
    System.out.println("1: Name | 2: Dept | 3: Credits");
    int choice = sc.nextInt(); sc.nextLine();
    String col = choice == 1 ? "name" : (choice == 2 ? "dept_name" : "tot_cred");
    System.out.print("Enter new value: "); String val = sc.nextLine();
    String sql = "UPDATE student SET " + col + " = ? WHERE ID = ?";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, val); pstmt.setString(2, id);
        pstmt.executeUpdate();
    }
}
}

```

```

private static void showInstructor(Connection conn, String id) throws SQLException {
    try (PreparedStatement pstmt = conn.prepareStatement("SELECT * FROM instructor
WHERE ID = ?")) {
        pstmt.setString(1, id);
        ResultSet rs = pstmt.executeQuery();
    }
}

```

```

        if (rs.next()) System.out.printf("ID: %s | Name: %s | Salary: %.2f\n", rs.getString("ID"),
rs.getString("name"), rs.getDouble("salary"));

        else System.out.println("Not found.");

    }

}

```

```

private static void showCourseEnrollment(Connection conn) throws SQLException {

    System.out.print("Enter Course ID: "); String cid = sc.nextLine();

    String sql = "SELECT s.ID, s.name, t.semester, t.year FROM takes t JOIN student s ON
t.ID = s.ID WHERE t.course_id = ?";

    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, cid);

        ResultSet rs = pstmt.executeQuery();

        while (rs.next()) System.out.printf("ID: %s | Name: %s | %s %d\n", rs.getString("ID"),
rs.getString("name"), rs.getString("semester"), rs.getInt("year"));

    }

}

```

```

private static void showInstructorCourses(Connection conn) throws SQLException {

    System.out.print("Enter Instructor ID: "); String iid = sc.nextLine();

    String sql = "SELECT course_id FROM teaches WHERE ID = ?";

    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, iid);

        ResultSet rs = pstmt.executeQuery();

        while (rs.next()) System.out.println("Taught Course: " + rs.getString("course_id"));

    }

}

```

```

private static void updateSalary(Connection conn, String type) throws SQLException {

```

```
System.out.print("Enter Instructor ID: "); String id = sc.nextLine();  
double multiplier = type.equals("HRA") ? 1.15 : 0.80;  
try (PreparedStatement pstmt = conn.prepareStatement("UPDATE instructor SET salary =  
salary * ? WHERE ID = ?")) {  
    pstmt.setDouble(1, multiplier); pstmt.setString(2, id);  
    if (pstmt.executeUpdate() > 0) {  
        System.out.println(type + " processed.");  
        showInstructor(conn, id);  
    }  
}  
}
```